

---

# Práctica 4 - Diseño y simulación de un reconocedor de secuencias en VHDL

Alejandro Otero Vázquez

13 de enero de 2025

---

En este documento se detalla la metodología para el diseño e implementación de un reconocedor de secuencias en lenguaje VHDL.

El sistema recibirá un bit por cada ciclo de reloj. Una señal binaria Z tomará valor 1 solamente cuando los cuatro últimos bits de entrada formen una secuencia 1101 y valdrá 0 en el resto de casos. Una señal asincrónica reset devolverá el sistema a su estado inicial.

## 1. Modelado usando una FSM

Una forma de modelar este reconocedor de secuencias es a partir de una máquina de estados. Esta máquina de estados presentará 4 estados conectados por la lógica que se le dé al sistema. Se definen 3 procesos, uno de ellos (sm) determina el estado siguiente y la salida en función del estado actual y la señal de entrada. El segundo proceso (state reg) modela el registro de estado con la entrada reset asincrónica y la señal de reloj. Se define para esto una variable que contendrá los posibles estados.

Se definen las señales de entrada y salida binarias del siguiente modo:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity machine_moore is
    Port ( X : in  STD_ULOGIC;
          Clk : in  STD_LOGIC;
          Reset : in  STD_ULOGIC;
          Z : out  STD_ULOGIC);
end machine_moore;
```

Donde X es el bit de entrada que se introduce al reconocedor en cada ciclo de reloj. Z será la salida binaria que se activará cuando el sistema reconozca la secuencia.

### 1.1. VHDL Module

La arquitectura de la entidad **machine\_moore**, requiere de una serie de señales auxiliares que serán todas de tipo state, se guardará en cada señal respectivamente el estado actual y el siguiente. Además, se definen los dos procesos que se han descrito al inicio.

```
architecture Behavioral_moore of machine_moore is
    type state is (s0,s1,s2,s3);
    signal current_state: state;
    signal next_state: state;
begin
    sm: process (current_state,x)
```

---

```

begin
    next_state <= current_state;
    case current_state is
        when s0 =>
            Z <= '0' after 100 ns;
            if (X='1') then next_state <= s1;
            end if;
        when s1 =>
            if (X='1') then next_state <= s2;
            else next_state <= s0;
            end if;
        when s2 =>
            if (X='0') then next_state <= s3;
            end if;
        when s3 =>
            if (X='1') then next_state <= s1;
            Z <= '1' after 100 ns, '0' after 200 ns;
            else next_state <= s0;
            end if;
    end case;
end process sm;

state_reg: process (clk, reset)
begin
    if reset = '1' then
        current_state <= s0;
    elsif (rising_edge(clk)) then
        current_state <= next_state;
    end if;
end process state_reg;
end Behavioral_moore;

```

En el process sm aparece un case en el que se especifica la lógica que gobernará como se pasa de un estado a otro. En el último estado, se asigna un 1 a la variable de salida si la secuencia se completa. Esta asignación se mantiene en espera 100 ns. En las condiciones del sistema se establece que la señal debe cambiar un ciclo después de que se haya completado la secuencia. Como cada ciclo de reloj serán 100 ns la señal de salida variará con un ciclo de retraso. Lo mismo ocurre en el estado s0, la asignación de Z a 0 se pospone un ciclo para que si el sistema vuelve a ese estado justo un ciclo después a que se haya completado la secuencia no se solapen las asignaciones.

---

## 1.2. VHDL Test Bench

En el archivo del test bench se especifican los valores que le daremos a la señal de entrada durante la simulación. Se simulará una secuencia de '01101101010111' durante 1400 ns.

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
ENTITY moore_tb IS
END moore_tb;
ARCHITECTURE behavior OF moore_tb IS
    COMPONENT machine_moore
    PORT(
        X : IN  std_logic;
        Clk : IN  std_logic;
        Reset : IN  std_logic;
        Z : OUT  std_logic
    );
    END COMPONENT;

    signal X : std_logic := '0';
    signal Clk : std_logic := '0';
    signal Reset : std_logic := '1';

    signal Z : std_logic;

BEGIN

    uut: machine_moore PORT MAP (
        X => X,
        Clk => Clk,
        Reset => Reset,
        Z => Z
    );
    X <= '1' after 100 ns, '0' after 300 ns, '1' after 400 ns,
    '0' after 600 ns, '1' after 700 ns, '0' after 800 ns, '1' after 900 ns,
    '0' after 1000 ns, '1' after 1100 ns;
    Reset <= '0' after 100 ns;
    Clk_process : process
    begin
        Clk <= '0';
        wait for 100 ns/2;
        Clk <= '1';
        wait for 100 ns/2;
    end process;
END;
```

Al tratarse de una lógica secuencial se define una señal de reloj y un periodo de reloj (100ns) en el Test bench. Cabe destacar que la lista de valores que se asigna en este documento será común a todas las implementaciones, de modo que este apartado se referenciará de aquí en adelante.

### 1.3. ISIM Simulation

Si se simula la salida para la lista de valores del test bench se obtiene el siguiente diagrama. En él, se observa que la salida es correcta en toda la simulación. Además se verifica que la asignación de reconocimiento se realiza un ciclo de reloj más tarde.

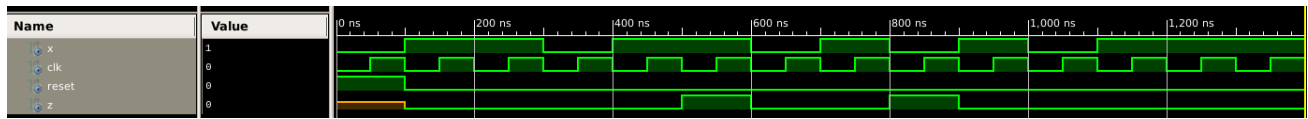


Figura 1: Simulación de una máquina de estados síncrona.

### 1.4. Synthesize & Schematic

Desde la herramienta View RTL Svhematic y View Technology Schematic podemos ver como implementa el programa a partir de puertas lógicas.

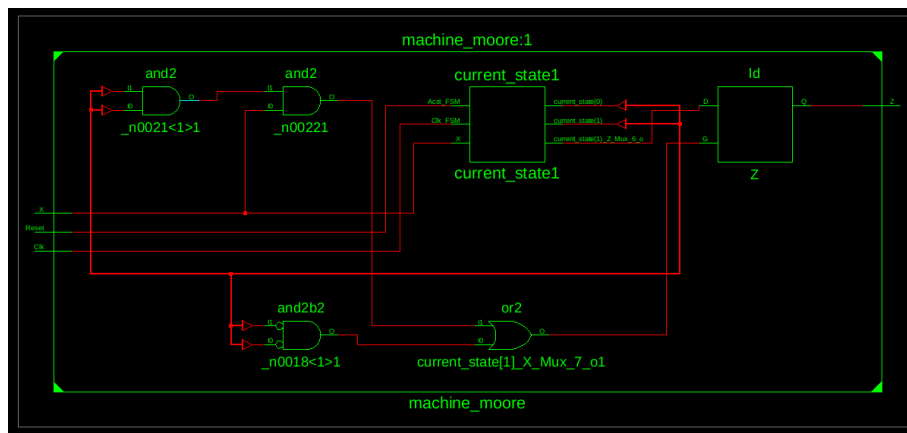


Figura 2: Esquemático de una máquina de estados mediante la herramienta View RTL Svhematic.

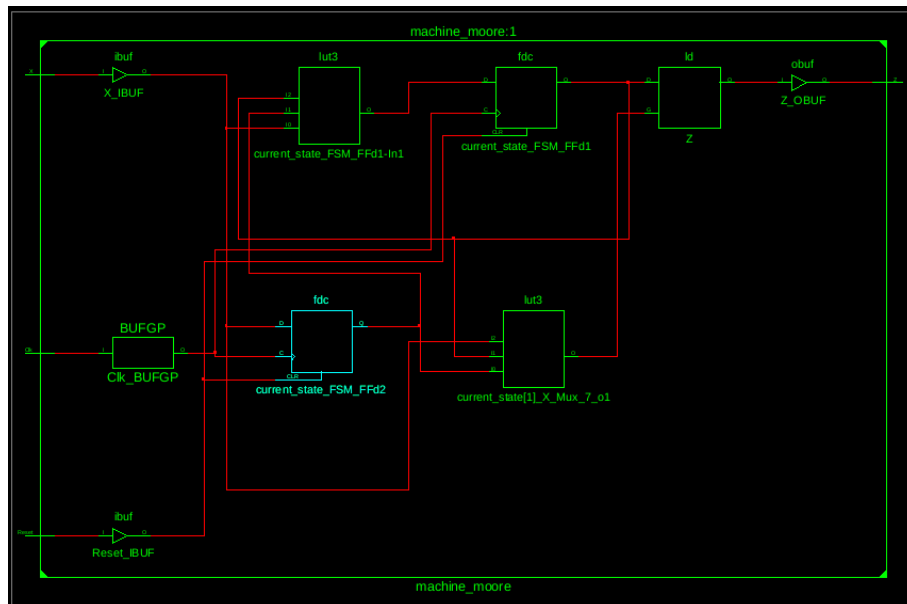


Figura 3: Esquemático de una máquina de estados mediante la herramienta View Technology Svhematic

---

## 2. Modelado usando una ROM

Una forma de modelar este reconocedor de secuencias es a partir de una memoria ROM. La memoria ROM tendrá 16 direcciones de memoria, en cada una de las cuales que se guardará un dato de 4 bits. Esta memoria ROM devolverá 3 bits que serán los 3 últimos valores de la señal de entrada que han entrado al sistema y un cuarto bit con la salida binaria de reconocimiento de patrón.

Se definen las señales de entrada y salida binarias del siguiente modo:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
entity ROM_entity is
    Port ( X : in  STD_LOGIC;
          Clk : in  STD_LOGIC;
          Reset : in  STD_LOGIC;
          Z : out  STD_LOGIC);
end ROM_entity;
```

Donde X es el bit de entrada que se introduce al reconocedor en cada ciclo de reloj. Z será la salida binaria que se activará cuando el sistema reconozca la secuencia.

### 2.1. VHDL Module

La arquitectura de la entidad **ROM\_entity**, requiere de una serie de señales auxiliares. La variable ROM\_content contendrá los 4 bits que se almacenarán en cada una de las direcciones de memoria. La rom accederá a la dirección de memoria '1001' cuando la secuencia de entrada de los últimos cuatro bits haya sido esa. La información almacenada ahí de memoria deberá contener los tres ultimos bits que se han introducido al sistema '001' y la salida Z, es decir, '0010'. Operando así con todas las direcciones de memoria se llega al array ROM\_content. Si se fija, tansolo la dirección de memoria '1101' contiene un dato almacenado que termine con un 1.

```
architecture Behavioral of ROM_entity is
    type memory is array (0 to 15) of std_logic_vector(3 downto 0);
    constant ROM_content: memory := (
        — Modificacion del contenido de la memoria ROM
        "0000", "0010", "0100", "0110",
        "1000", "1010", "1100", "1110",
        "0000", "0010", "0100", "0110",
        "1000", "1011", "1100", "1110" — Cambio en el registro '1101'
    );
    signal adres: std_logic_vector(3 downto 0) := (others => '0');
begin
    synch_rom: process (Clk)
    begin
        if (reset = '1') then adres <= "0000";
        elsif (falling_edge(Clk)) then
            adres <= adres(2 downto 0) & X;
            Z <= ROM_content(conv_integer(adres))(0);
        end if;
    end process synch_rom;
end Behavioral;
```

Cuando la señal de reloj pasa de 1 a 0 se actualiza la dirección de memoria addres con los últimos 4 bits de entrada. Z será el ultimo bit de el contenido almacenado en esa dirección de memoria. Se actualiza la señal un periodo después a que se haya verificado la secuencia.

## 2.2. VHDL Test Bench

En el archivo del test bench se especifican los valores que le daremos a la señal de entrada durante la simulación. Se simulará una secuencia de '011011010111' durante 1400 ns.

Al tratarse de una lógica secuencial se define una señal de reloj y un periodo de reloj (100ns) en el Test bench.

## 2.3. ISIM Simulation

Si se simula la salida para la lista de valores del test bench se obtiene el siguiente diagrama. En él, se observa que la salida es correcta en toda la simulación. Además se verifica que la asignación de reconocimiento se realiza un ciclo de reloj más tarde.

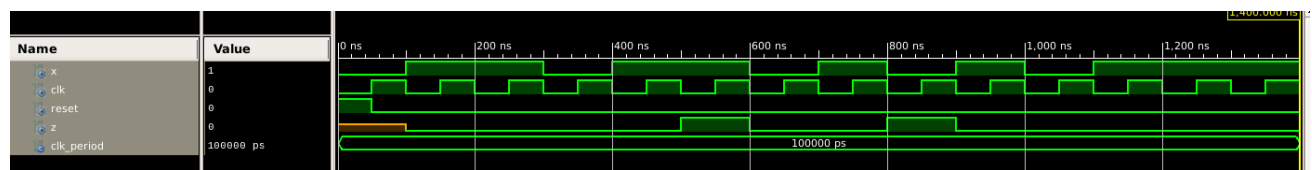


Figura 4: Simulación de una memoria rom.

## 2.4. Synthesize & Schematic

Desde la herramienta View RTL Svchematic y View Technology Schematic podemos ver como implementa el programa a partir de puertas lógicas.

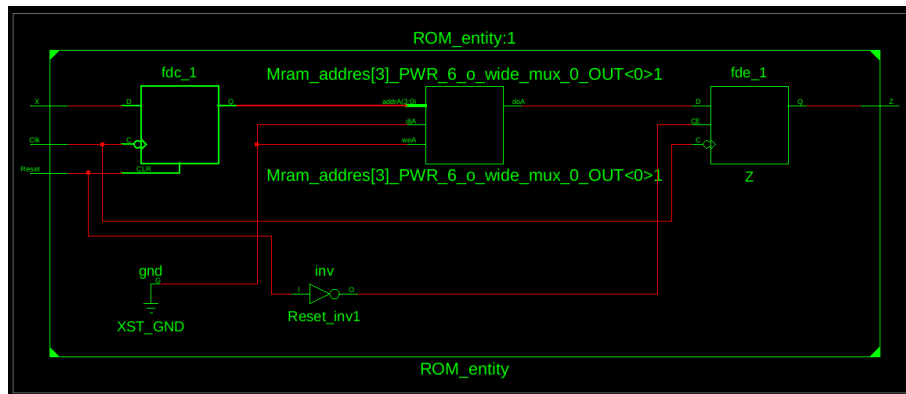


Figura 5: Esquemático de una memoria ROM mediante la herramienta View RTL Svchematic.

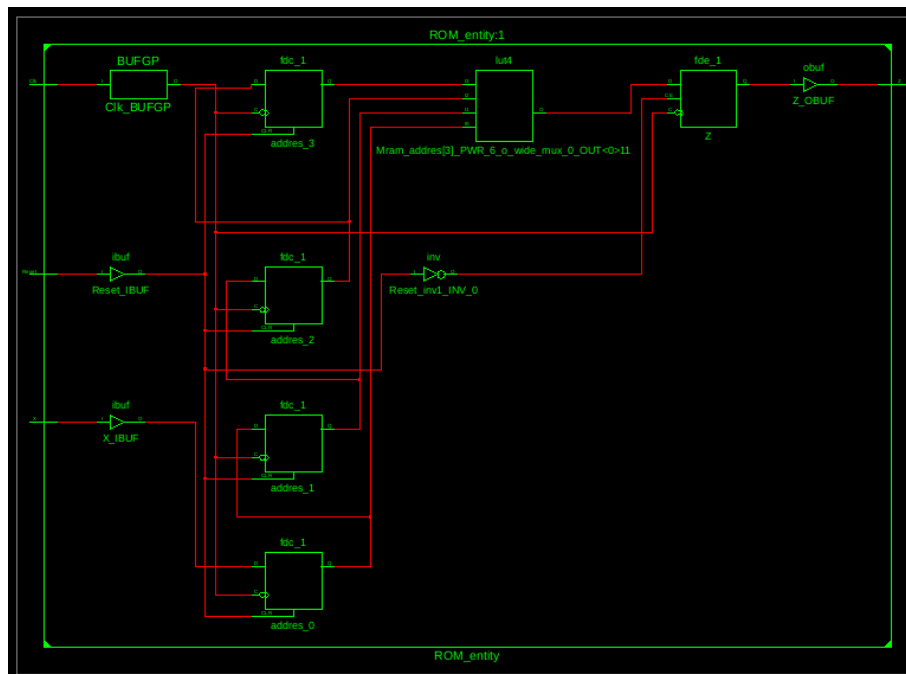


Figura 6: Esquemático de una memoria ROM mediante la herramienta View Technology Svchematic

---

### 3. Modelado usando un salvado de registro

Aunque la práctica no lo pide, he modelado el reconocedor de secuencias de una forma que resume en gran medida las secuencias que debe seguir el programa. Básicamente este modelo se basa en guardar el registro de datos de entrada en una variable de 4 bits que se va actualizando en cada ciclo de reloj. Cuando esta variable contiene la secuencia deseada la salida Z se hace 1.

Se definen las señales de entrada y salida binarias del siguiente modo:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity machine_ROM is
    Port ( X : in  STD_ULOGIC;
          Clk : in  STD_ULOGIC;
          Reset : in  STD_ULOGIC;
          Z : out  STD_ULOGIC);
end machine_ROM;
```

Donde X es el bit de entrada que se introduce al reconocedor en cada ciclo de reloj. Z será la salida binaria que se activará cuando el sistema reconozca la secuencia.

#### 3.1. VHDL Module

La arquitectura de la entidad **machine\_ROM**, requiere de una señal auxiliar que será un vector de 4 bits.

```
architecture Behavioral_ROM of machine_ROM is
    signal state_register: std_logic_vector(3 downto 0) := (others => '0');
begin
    rom_m: process (Clk)
    begin
        if falling_edge(Clk) then
            state_register <= state_register(2 downto 0) & X;
            if (state_register = "1101") then
                Z <= '1';
            else
                Z <= '0';
            end if;
        end if;
    end process rom_m;
end Behavioral_ROM;
```

Cuando la señal de reloj pasa de 1 a 0 se actualiza la señal auxiliar con los últimos 4 bits de entrada.

#### 3.2. VHDL Test Bench

En el archivo del test bench se especifican los valores que le daremos a la señal de entrada durante la simulación. Se simulará una secuencia de '0110110101011' durante 1400 ns.

Al tratarse de una lógica secuencial se define una señal de reloj y un periodo de reloj (100ns) en el Test bench.



### 3.3. ISIM Simulation

Si se simula la salida para la lista de valores del test bench se obtiene el siguiente diagrama. En él, se observa que la salida es correcta en toda la simulación. Además se verifica que la asignación de reconocimiento se realiza un ciclo de reloj más tarde.

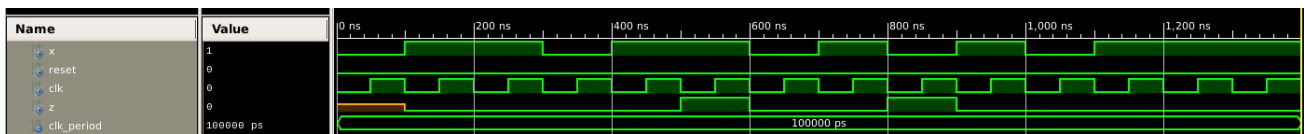


Figura 7: Simulación de un reconocedor de secuencias.

### 3.4. Synthesize & Schematic

Desde la herramienta View RTL Svhematic y View Technology Schematic podemos ver como implementa el programa a partir de puertas lógicas.

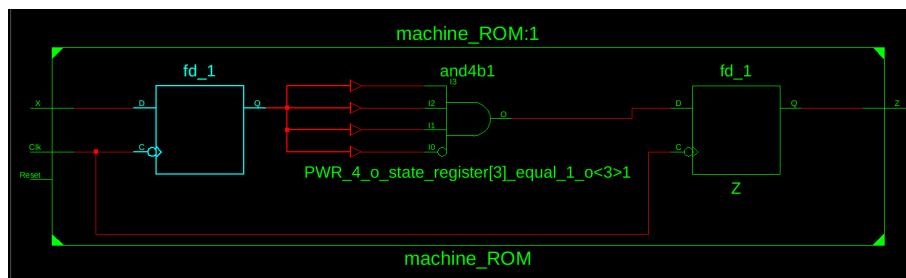


Figura 8: Esquemático de un reconocedor de secuencias mediante la herramienta View RTL Svhematic.

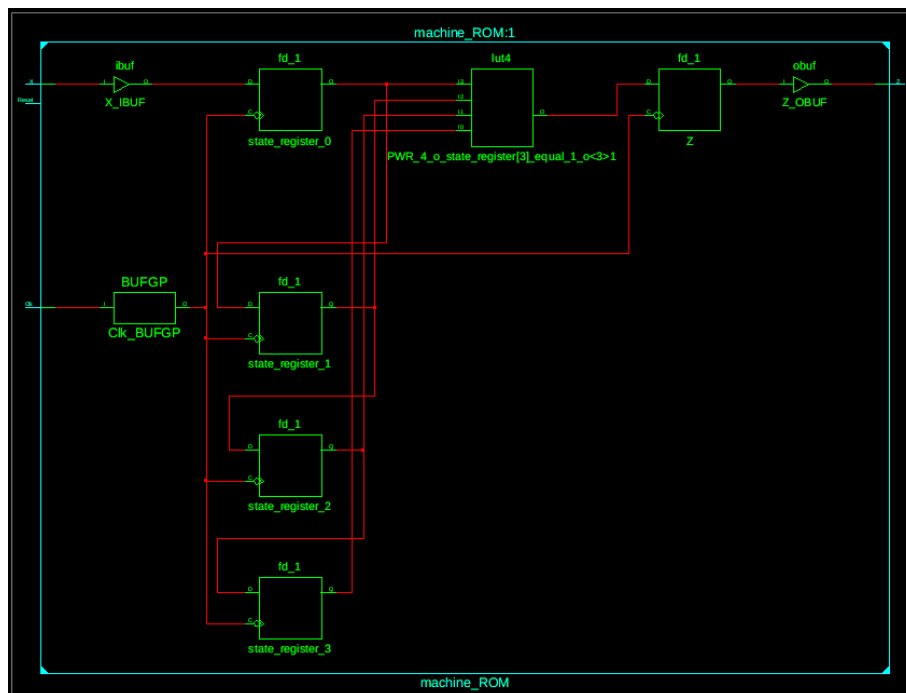


Figura 9: Esquemático de un reconocedor de secuencias mediante la herramienta View Technology Svhematic